IdempotentWallet

Study Guide in Simple English (IT3133)

A simpler version of the full study guide. Same topics, same order, easier words.

1. The story in one page

> **In one sentence:** We built a digital wallet where pressing "Send" five times still moves the money only once. We prove it by breaking the system on purpose in front of the examiners.

What the lecturer is testing. The module is *Programming Distributed Components*. The big idea: when programs talk over a network, things fail halfway, and the same request can arrive twice. The cure (Distributed Design, slide 4) is: **send messages at-least-once, and make the receiver idempotent using a stable operation ID.** Our project is a working proof of that sentence.

The story in five acts

- **1. The problem.** A customer taps "Send LKR 500". The Wi-Fi drops. The customer taps again. Did the money move once or twice? The server cannot tell a lost request from a lost reply.
- **2. The rule.** Never overwrite a balance. Only add a new line to a permanent statement (an "event"). This is **event sourcing**. It gives a full audit trail.
- **3. The guard.** Every money request carries an **Idempotency-Key**. The database remembers key -> result. A repeat gets the saved receipt, not a second debit.
- **4. The pipeline.** Events leave the database through an **outbox**, travel through RabbitMQ, and the receiver skips duplicates using an **inbox**. A transfer between two wallets is a **saga** with a refund if step two fails.
- **5. The proof.** We cause failures on purpose (lost replies, duplicate messages, a crashed service, a flaky payment provider) and show that money is never created or lost by a retry.

Why "Exactly-Once(ish)"? Perfect exactly-once delivery over a network is impossible. What we build is **effectively-once**: a message may arrive many times, but its effect happens once. The "ish" is the honest word for that gap. Saying it confidently in the viva earns marks.

What is in the project

Piece	What it is	Port
wallet-service	Wallets, ledger, idempotent API, outbox relay, consumers, mock payment provider	8080
transfer-service	The saga that moves money between two wallets	8081
PostgreSQL	Two databases: wallet and transfers	5432
RabbitMQ	The message broker between the services	5672 (UI 15672)
Tests	Unit tests, end-to-end tests, a k6 load script	src/test, k6/
Demo files	requests.http files and the README	project folders

In numbers: 2 services, 2 databases, 1 broker, 3 idempotency layers, 11 demo scenarios.

Where the code has a weakness, this guide says so (section 13). Examiners reward students who name their own weak spots.

2. Foundations: the ideas under everything

2.1 Partial failure

A distributed system is several programs on different machines sending messages to each other. Its main problem is **partial failure**: one part can fail while the others keep working. *Analogy:* texting a friend. "Delivered" does not mean "read", and silence does not mean "no".

2.2 The lost-reply problem (the heart of the project)

A client sends "transfer LKR 500" and hears nothing. There are three possible worlds, and the client cannot tell which one it is in:

- The request never arrived. Money did not move.
- The server crashed before saving. Money did not move.
- The server saved the transfer, but the reply was lost. Money moved.

The only safe choice is to **retry**. So retries are required, and duplicates are unavoidable. The whole project exists to make a duplicate harmless. (This is like the Two Generals problem: two parties on an unreliable link can never be fully sure the other got the last message.)

2.3 Delivery guarantees

Guarantee	Meaning	Weakness
At-most-once	Delivered 0 or 1 times. Send once, never retry.	Messages can be lost
At-least-once	Delivered 1 or more times. Retry until acknowledged.	Messages can be duplicated
Exactly-once	Delivered exactly 1 time.	Impossible over a network in general

Our recipe: use at-least-once (nothing is lost), then make the receiver idempotent (duplicates change nothing). Result: effectively-once.

2.4 Idempotency

An operation is **idempotent** if doing it many times has the same effect as doing it once. *Analogy:* a lift button. Pressing it five times still calls one lift. A light switch labelled "set to ON" is idempotent. One labelled "toggle" is not.

Operation	Idempotent?	Why
Set balance to 500	Yes	Same end state however often you run it
Add 500 to balance	No	Every run changes the result
HTTP GET, PUT, DELETE	Yes (by specification)	They describe a state, not an increment
HTTP POST (create a transfer)	No by default	Each call creates something new

Money movements are "add 500" operations that arrive by POST, so they are *not* idempotent naturally. We make them idempotent: give each **intention** a stable ID (the Idempotency-Key) and store key -> result. The key belongs to the intention ("this one transfer"), not to one network attempt, so every retry carries the same key.

2.5 CAP and eventual consistency

CAP theorem: during a network partition you must choose between Consistency (everyone sees the latest write) and Availability (everyone gets an answer). Partition tolerance is not optional. **Our choice:** ledger writes favour consistency per wallet (a conflicting write is rejected and retried). Between services

(notifications, saga replies) the system is **eventually consistent**: the other side catches up a moment later.

2.6 Ordering without clocks

The lectures cover Lamport and vector clocks. We do not need them: each wallet is owned by one service and has its own counter `seq` (1, 2, 3...). One order per wallet is enough. Choosing the simplest tool that works is a design skill worth mentioning.

2.7 Where each idea comes from in the slides

Idea	Lecture	Used in the project as
Partial failure, retries, backoff	01 Introduction, slide 24	Retry with jitter, timeouts
CAP, eventual consistency	03 Distributed Programming	Consistency choices above
Idempotency and Exactly-Once(ish)	04 Distributed Design, slides 4-5	The three idempotency layers
Outbox, Saga, Resilience, CQRS	04 Distributed Design, slides 3-7	Sections 5 to 9
Message middleware	05 Middleware	RabbitMQ
Race conditions, deadlocks	06 Parallel Processing	Optimistic concurrency, parallel tests
Stubs, mocks, boundaries	07 Component Testing	Tests (section 10)

3. Architecture

Clients call either REST API. The two services **never call each other directly** and **never read each other's tables**. They only exchange commands and replies through RabbitMQ.

Why two services? The module is about distributed components. Two services with separate databases is the smallest design that gives a real network boundary, so we get a real saga, real duplicate messages and real partial failures.

Design rules

- **Database per service.** No shared tables means no hidden coupling.
- **Messages for anything multi-step.** If one service is down, messages wait in the broker.
- **Idempotent at every boundary.** Every place a request or message can repeat has a duplicate guard.
- **One writer per wallet.** Only wallet-service writes a wallet's ledger, so ordering is simple.

Flow A: a top-up (one service)

- Client sends `POST /wallets/{id}/topups` with an Idempotency-Key.
- TraceFilter gives the request a trace ID.
- IdempotentExecutor opens a database transaction and claims the key.
- LedgerService adds a *Credited* event, updates the balance row, and writes an outbox row, all in the same transaction.
- The transaction commits and the receipt is saved against the key. Client gets **201**.
- OutboxRelay (runs every second) publishes the outbox row to RabbitMQ with a stable message ID.
- NotificationConsumer saves the message ID in the inbox and writes a notification in one transaction. A repeated message is skipped.

Flow B: a transfer (two services)

- Client sends `POST /transfers`. Transfer-service saves it as PENDING plus a DEBIT command in its outbox, then answers **202 Accepted**.

- The relay publishes the command. RabbitMQ sends it to the wallet service's command queue.
- CommandConsumer debits the source wallet using the key `transferId:DEBIT` and writes a reply to its outbox.
- The wallet relay publishes the reply. Transfer-service's ReplyConsumer receives it.
- The saga moves PENDING -> DEBITED and queues CREDIT. Success ends in COMPLETED. A failed credit triggers REFUND and ends in COMPENSATED.

Who does what

Component	Job	Idea shown
WalletController, CardTopUpController	Public REST API	Idempotent API design
IdempotentExecutor	Claims the key, runs the effect, saves the receipt, in one transaction	Idempotency layer 1
LedgerService	Adds events, updates balance, writes outbox row, replays history	Event sourcing, optimistic concurrency
OutboxRelay	Publishes outbox rows to RabbitMQ	Transactional outbox
NotificationConsumer	Applies an event once using the inbox	Idempotency layer 2
CommandConsumer	Runs saga commands once per transferId:step	Idempotent commands
ProviderClient, MockProviderController	Calls an outside payment provider safely	Idempotency layer 3, resilience
SagaService, TransferController, ReplyConsumer	Runs the transfer state machine with refunds and recovery	Saga, compensation
ChaosController, ChaosState	Switches that break things on purpose	Fault injection
TraceFilter	One trace ID per request	Observability
Flyway V1__init.sql	Creates every table, versioned	Database migration
docker-compose.yml	Starts PostgreSQL and RabbitMQ with one command	Repeatable environment

4. Data layer: PostgreSQL and the tables

The database is not just storage. It is the **referee**. It can say "no" atomically even when many requests collide.

4.1 Transactions and ACID

A **transaction** is a group of changes that succeed or fail together. ACID gives four promises:

Letter	Promise	Analogy	Where we use it
Atomicity	All changes happen, or none	Bank transfer: debit and credit both, or neither	Key row + ledger event + outbox row commit together
Consistency	Rules are never broken	A form that refuses a blank required field	amount_minor > 0, unique (wallet_id, seq)
Isolation	Concurrent transactions do not see each other's half-done work	Two cashiers with separate notepads	Duplicate requests are lined up by the unique index
Durability	Once committed, it survives a crash	Ink, not pencil	A committed event is never lost

Isolation level. PostgreSQL defaults to READ COMMITTED: each statement sees only data committed before it began. When two identical requests insert the same key, the second INSERT *waits* for the first to finish. If the first commits, the second sees the row and replays it. If the first rolls back, the second becomes the owner.

4.2 Constraints: rules the database enforces

- **PRIMARY KEY / UNIQUE:** no two rows may share the value. This is our doorman.
- **CHECK:** a rule on values, for example amount_minor > 0.
- **FOREIGN KEY:** a row must point to a real parent (an event needs a real wallet).
- **ON CONFLICT DO NOTHING:** "try to insert; if the key exists, do nothing and tell me zero rows changed". This is how we safely ask the doorman.

4.3 Why money is a bigint, never a decimal

We store **minor units** (LKR 10.00 is stored as 1000). Floating-point numbers cannot store values like 0.1 exactly, so sums drift (0.1 + 0.2 gives 0.30000000000000004). Whole numbers never drift.

4.4 Every table and why it exists

Table	DB	Purpose	Key detail
wallets	wallet	One row per wallet	Parent for events and balance
ledger_events	wallet	Source of truth: every credit and debit, append-only	UNIQUE (wallet_id, seq) plus a trigger that forbids update and delete
wallet_balance	wallet	Read model: current balance and last seq	Updated only WHERE last_seq = ? (optimistic concurrency)
idempotency_keys	both	Layer 1: key, request hash, saved response, expiry	Primary key (scope, idem_key)
outbox	both	Messages waiting to be published	Same transaction as the data
inbox	wallet	Layer 2: message IDs already processed	Primary key (consumer, message_id)
notification_log	wallet	Proof of the consumer's side effect	UNIQUE (message_id): one row even if delivered twice
transfers	transfers	Saga state, one row per transfer	The state column is the state machine

4.5 Three SQL ideas to know by heart

1. Append-only ledger, enforced by the database. Even a buggy program cannot rewrite history:

```
CREATE TRIGGER ledger_events_immutable
BEFORE UPDATE OR DELETE ON ledger_events
FOR EACH ROW EXECUTE FUNCTION forbid_ledger_change();
```

2. The doorman. The unique index plus ON CONFLICT DO NOTHING decides who owns a key:

```
INSERT INTO idempotency_keys (scope, idem_key, request_hash,
                              status, expires_at)
VALUES (?, ?, ?, 'IN_PROGRESS', now() + interval '24 hours')
ON CONFLICT (scope, idem_key) DO NOTHING
```

3. Several relays without double publishing. SKIP LOCKED makes a second worker skip rows another worker already holds:

```
SELECT id, event_id, topic, payload FROM outbox
WHERE status = 'PENDING' ORDER BY id LIMIT 50
FOR UPDATE SKIP LOCKED
```

4.6 Flyway migrations

Flyway runs numbered SQL scripts (V1__init.sql, V2__...) in order and remembers which ones ran. *Analogy:* version control for the database structure. **Rule:** never edit a script that already ran. Add a new version instead. This keeps every team member's database identical.

5. Event sourcing and CQRS: the ledger

5.1 Event sourcing

Instead of storing the current state (balance = 600), store the **history of facts** that made it (credited 1000, debited 400). The balance is calculated by adding up the history. *Analogy:* a bank statement versus a whiteboard. A whiteboard shows only today's number, and wiping it destroys the past. A statement is a list of lines that are never erased.

Events are facts in the past tense: *Credited*, *Debited*. They are never edited. A mistake is fixed by adding a new compensating event, like a bank reversal.

Benefit	Cost
Full audit trail: who moved what and when	Reading the balance means adding up events (we avoid this with a stored balance)
Replay: rebuild the state at any time	The stored balance is a second copy that must stay in step
Fits messaging: each event can be published	Changing an event's shape later needs care
Bugs are recoverable: fix the code, replay the events	More concepts for the team to learn

5.2 CQRS

Command Query Responsibility Segregation: use one model for changing data and a different model for reading it. *Analogy:* the cashier's ledger book (careful, authoritative) versus the notice board in the lobby (quick to read, built from the ledger).

- **Write side:** ledger_events, the source of truth.
- **Read side:** wallet_balance, a projection built from the events.

Honest simplification: both live in one database and the projection updates in the same transaction, so it is always up to date. Real CQRS usually updates the read side later (eventual consistency). Say this in the viva.

5.3 How LedgerService.append works

- Read the current balance and last_seq.
- Check the rules. A debit bigger than the balance throws InsufficientFundsException.
- Compute seq = last_seq + 1 and insert the event. The unique (wallet_id, seq) stops a second writer from taking the same number.
- Update the balance row only if last_seq is still what we read.
- Insert the outbox row so the event is published later.

```
long seq = current.lastSeq() + 1;
jdbc.update("INSERT INTO ledger_events (...) VALUES (...)", ...);
int updated = jdbc.update(
  "UPDATE wallet_balance SET balance_minor = ?, last_seq = ? "
  + "WHERE wallet_id = ? AND last_seq = ?", ...);
if (updated == 0)
    throw new DomainErrors.OptimisticConflictException();
```

`Propagation.MANDATORY` means the method refuses to run unless a transaction is already open. So nobody can write an event outside the idempotent transaction.

5.4 Optimistic concurrency

Instead of locking a record so others must wait (**pessimistic**), let everyone proceed and detect a clash at the moment of writing (**optimistic**). The loser retries on fresh data. *Analogy:* two people editing a shared document. On saving, the second is told "someone changed it, reload and redo".

Worked example: the classic overdraft race

A wallet holds 100. Two withdrawals of 80 arrive at the same instant.

- Both read balance 100 and last_seq 1. Both think 80 is fine.
- Both try to insert event seq = 2. The unique index lets one in. The other fails with a duplicate-key error.
- The loser's transaction rolls back. ConflictRetry waits a few random milliseconds and runs it again.
- On the retry it reads balance 20, so the withdrawal now fails with INSUFFICIENT_FUNDS.

Without the unique constraint, both would succeed and the wallet would be overdrawn. This is the race condition from the Parallel Processing lecture, solved without locks held across wallets, so there is no circular wait and no deadlock.

5.5 Replay: proof the read model is disposable

`GET /wallets/{id}/balance/replay` recomputes the balance only from events and returns both numbers with `"consistent": true` if they match. **Not implemented:** snapshots (saving the balance every N events so replay stays fast). Mention as future work.

6. Idempotency in depth (the marks section)

This is the section the lecturer cares about most. Know it well enough to draw it on a whiteboard.

6.1 Three layers, three different duplicate sources

Layer	Where duplicates come from	Guard	Code
1. Client to API	Users double-click, apps retry after a timeout	Idempotency-Key header, idempotency_keys table	IdempotentExecutor
2. Broker to consumer	RabbitMQ delivers at-least-once; the relay can publish twice	inbox table keyed by message ID	NotificationConsumer, CommandConsumer
3. Service to outside provider	Our own retries call the provider again	We send a stable key; the provider returns the same charge	ProviderClient, MockProviderController

One guard is not enough, because each boundary is a different place where a message can repeat.

6.2 Layer 1 step by step: IdempotentExecutor

Plain English: before doing anything, the server writes the client's key into a table. The unique index allows only one writer per key. The winner does the work and files the receipt. Everyone else is handed the receipt.

```
@Transactional
public Stored execute(scope, key, requestHash, Supplier<Stored> effect) {
  int inserted = jdbc.update("INSERT INTO idempotency_keys ...
                             ON CONFLICT DO NOTHING", ...);
  if (inserted == 0) {                // somebody already owns the key
    Existing e = loadExisting(scope, key);
    if (!e.requestHash().equals(requestHash))
        throw new KeyReuseException(); // 422
    return new Stored(e.code(), e.body(), true);   // replay
  }
  Stored result = effect.get();   // ledger append + outbox, SAME tx
```

```
    jdbc.update("UPDATE idempotency_keys SET status='COMPLETED', ...");
    return result;
}
```

Situation	What happens	Client sees
First time	Insert wins, effect runs, receipt saved	201, Idempotent-Replayed: false
Same key, same payload, later	Insert returns 0 rows, hashes match, saved receipt returned	Same body, Idempotent-Replayed: true
Same key, different payload	Hashes differ	422 IDEMPOTENCY_KEY_REUSED
Same key at the same instant	Second insert waits for the first transaction, then acts like row 2	Idempotent-Replayed: true
First attempt rolled back	Its key row vanished, so the waiting request becomes the owner	Normal success
Key older than 24 hours	Expired row deleted, request treated as new	New effect
No key at all	Rejected before the executor runs	400 MISSING_IDEMPOTENCY_KEY

6.3 Why the key and the effect must commit together

There are two ways to get this wrong. One transaction closes both:

- **Effect saved, key not saved:** a retry finds no key and repeats the effect. Double debit.
- **Key saved, effect not saved:** a retry finds the key and replays a "success" for money that never moved. Silent loss.

Because the key row, the ledger event and the outbox row are written in one transaction, they all exist or none do.

6.4 Design details examiners ask about

- **Key scope.** The key is stored with a scope like POST /wallets/{id}/topups, so the same key on a different endpoint or wallet is a different operation.
- **Request hash.** A SHA-256 of a standard string (scope and amount). It catches a client reusing a key for a different request.
- **TTL.** Keys expire after 24 hours and a scheduled job (IdempotencyCleanup) deletes old rows. After expiry a very late retry counts as new, so the TTL must be longer than any realistic client retry window.
- **Idempotent-Replayed header.** Tells the client that this answer came from the saved receipt.
- **Rejected requests are not remembered.** If the effect throws (e.g. insufficient funds), the transaction rolls back and the key row disappears, so a retry is judged fresh. That is deliberate for user calls because the balance may have changed. In the saga's CommandConsumer the opposite is chosen: a business failure is stored as FAILED, so every repeat gets the same answer.

6.5 Why the retry loop sits outside the transaction

ConflictRetry wraps the call to executor.execute(...). A transaction that hit a duplicate-key error is already poisoned and must roll back. Each retry needs a brand-new transaction, which only happens if the loop is outside the transactional method. The random delay (jitter) stops competing requests from retrying in step.

6.6 Walk-through: 50 identical requests at once

- All 50 try to insert the same key. The unique index lets one through. The other 49 wait.
- The winner adds one event, updates the balance and commits.

- Each waiting request wakes up, finds the committed row with a matching hash, and returns the saved receipt.
- Result: one ledger event, balance +1000, and 49 responses with Idempotent-Replayed: true. The counter wallet.duplicates.suppressed{layer=api} reads 49.

The e2e test and the k6 script check exactly this.

6.7 Layers 2 and 3

Layer 2 (inbox). The consumer first inserts the message ID into inbox with ON CONFLICT DO NOTHING. If zero rows were inserted, it is a duplicate: acknowledge and skip. The inbox row and the side effect share one transaction, for the same reason as in 6.3.

Layer 3 (outside provider). A card top-up first charges the mock provider using the key walletId:Idempotency-Key. The provider remembers keys, so a retry returns the same chargeId instead of charging again. Then we credit the wallet once through layer 1. **Honest limit:** a replayed card top-up still calls the provider before layer 1 notices the replay.

6.8 What idempotency does not do

It cannot make an effect *outside* our database safe unless the receiver also removes duplicates. A real SMS or email is at-least-once. We say this openly instead of claiming exactly-once.

7. Messaging: middleware, outbox, inbox, dead-letter queues

7.1 Message-oriented middleware and RabbitMQ

Middleware is software between applications so they can talk without knowing each other's details. Message-oriented middleware does this with messages that wait in queues. *Analogy:* a post office. You drop off a letter and leave. It waits safely until the recipient collects it. That is decoupling in time.

Term	Meaning	Analogy
Producer	Sends messages	The sender
Exchange	Receives messages and routes them by a routing key	The sorting office
Queue	Stores messages until consumed	A pigeonhole
Binding	Rule linking an exchange to a queue	A sorting rule
Consumer	Receives and processes messages	The recipient
Acknowledgement (ack)	"I processed it, you may delete it"	A signature on delivery
Durable	Survives a broker restart	Letters kept in a safe

Exchange	Routing keys	Queue	Used for
wallet.exchange	wallet.credited, wallet.debited	wallet.events.notification	Domain events to the notification consumer
wallet.exchange	reply.debit, reply.credit, reply.refund	transfer.replies	Saga replies back to transfer-service
transfer.exchange	cmd.debit, cmd.credit, cmd.refund	wallet.commands	Saga commands to wallet-service
wallet.dlx, transfer.dlx	queue-specific	*.dlq queues	Messages that could not be processed

7.2 The dual-write problem

A service must change its database *and* announce it on the broker. These are two separate systems, so you cannot commit both at once.

- Save, then publish: crash in between and the world never hears about the change.
- Publish, then save: crash in between and the world hears about a change that never happened.

Analogy: posting a letter and updating your diary as two separate acts. A phone call in the middle can leave them out of step.

7.3 The transactional outbox

Idea: write the message into an **outbox table** in the *same* database transaction as the data change. Now they are atomic. A separate worker, the **relay**, later reads pending rows and publishes them.

How our OutboxRelay works

- Every second it selects up to 50 PENDING rows with FOR UPDATE SKIP LOCKED.
- It publishes each row to RabbitMQ and sets the message ID to the row's event_id.
- It marks the row PUBLISHED.
- If publishing fails, it adds 1 to attempts. After 10 failures the row becomes DEAD so it cannot block the queue.
- If RabbitMQ is simply unreachable, it stops and leaves rows PENDING without counting an attempt (a broker outage is not the message's fault).

The outbox does not give exactly-once. If the relay crashes after publishing but before marking the row, it publishes again after restart. That is at-least-once, and it is exactly why the receiving side needs an inbox.

7.4 The inbox

The consumer saves each message ID before acting on it. A repeated ID means a duplicate, so it is acknowledged and skipped. In the demo you can switch on `duplicatePublish`, which makes the relay publish everything twice. The log says "Duplicate message ... skipped" and notification_log still holds one row per message.

7.5 Failure handling and the dead-letter queue (DLQ)

A **DLQ** holds messages that could not be processed after several attempts, so one bad message cannot block everything behind it. *Analogy:* the "return to sender" shelf at the post office.

- The listener retries a failing message 3 times with growing delays (0.5 s, then 1 s).
- `default-requeue-rejected: false` means that after the last failure the message is not put back.
- The queue has a dead-letter exchange, so the rejected message lands in the .dlq queue.
- A **poison message** always fails. The failConsumer chaos switch makes every message poison so you can show the DLQ filling up in the RabbitMQ web page.

Honest limit: nothing replays DLQ messages automatically. A human or a job must inspect them, fix the cause and re-publish. Call this operational follow-up.

7.6 Ordering

Outbox rows are published in id order and carry the aggregate's seq. Consumers do not rely on global order. The saga survives reordering because every state change is a guarded `UPDATE ... WHERE state = expected`.

8. Saga: transfers across two services

A transfer is a small **state machine** stored in the transfers table. Each reply from wallet-service moves it one step.

State	Meaning
PENDING	Debit command sent
DEBITED	Debit done, credit command sent
COMPLETED	Credit done. Money moved. (happy path)
FAILED	Debit failed (e.g. no funds). Nothing moved.
COMPENSATING	Credit failed, refund command sent
COMPENSATED	Refund done. Money returned.
MANUAL_REVIEW	Refund failed. A human must step in.

8.1 The problem

A transfer debits wallet A (in one database) and credits wallet B (possibly elsewhere). One database transaction cannot span two services. If the debit succeeds and the credit fails, money has vanished.

8.2 Two-phase commit (2PC) versus a saga

	Two-phase commit	Saga
Idea	A coordinator asks everyone to prepare, then orders commit or abort	A chain of local transactions, with a compensating action for each step
Atomicity	Strict, all or nothing	Eventual: in-between states are visible
Locks	Held across services while waiting	None held across services
If the coordinator dies	Participants can block, holding locks	State is in a database; recovery resumes it
Availability	Poor, every participant must be up	Good, messages wait in queues
Lecture view	Rejected for latency, availability and lock contention (slide 12)	Recommended (slides 6 and 11)

Analogy: booking a holiday. Flight, then hotel. If the hotel fails you do not "un-happen" the flight. You do a new action: cancel and refund the flight. That is compensation.

8.3 Orchestration versus choreography

- **Orchestration (our choice):** one coordinator (SagaService) tells each participant what to do next. Easy to see, trace and debug. Cost: a central component, which we protect by saving its state in the database.
- **Choreography:** services react to each other's events with no coordinator. Looser coupling, but the flow is scattered and harder to follow.

8.4 The transitions

Current state	Reply received	Guarded update	Next command
PENDING	DEBIT succeeded	PENDING -> DEBITED	CREDIT
PENDING	DEBIT failed	PENDING -> FAILED	none
DEBITED	CREDIT succeeded	DEBITED -> COMPLETED	none
DEBITED	CREDIT failed	DEBITED -> COMPENSATING	REFUND
COMPENSATING	REFUND succeeded	COMPENSATING -> COMPENSATED	none

Current state	Reply received	Guarded update	Next command
COMPENSATING	REFUND failed	COMPENSATING -> MANUAL_REVIEW	a human steps in

8.5 Why duplicates and reordering cannot break it

- **Replies are guarded.** A state change is `UPDATE transfers SET state='DEBITED' WHERE transfer_id=? AND state='PENDING'`. A duplicate or late reply changes 0 rows and is ignored. The next command is queued only when exactly one row changed.
- **Commands are idempotent.** Each command's key is transferId:step (e.g. ...:DEBIT). A repeated command is answered from the stored result and never applied twice.
- **The refund is a new fact, not an erase.** It adds a fresh Credited event on the source wallet. The ledger shows the debit and the refund: a clear audit trail.
- **The API is asynchronous.** POST /transfers answers 202 Accepted with a transferId. The client polls GET /transfers/{id}. The request itself is idempotent through its own key.

8.6 Crash recovery

`SagaService.resendStuck()` runs every 10 seconds. It finds transfers in PENDING, DEBITED or COMPENSATING that have not changed for 30 seconds and re-sends the command for the current step. Because commands are idempotent, re-sending is always safe.

What goes wrong	What saves us
transfer-service dies right after saving PENDING	The command is already in its outbox. The relay publishes it after restart.
wallet-service is down	The command waits in wallet.commands and is processed on restart.
Command lost or never processed	The recovery job re-sends it after 30 s. The wallet dedupes it.
A reply is processed twice	The guarded UPDATE changes 0 rows the second time.

Known gap (say it before they do). If a reply is lost after being published, the recovery job re-sends the command, but the wallet answers from its stored result and does not send a second reply, so the transfer can sit in DEBITED. Fix: when a command comes back as a replay, write the reply to the outbox again.

8.7 The in-between window

While a transfer is DEBITED, money has left A and not yet reached B. That is the price of a saga. The rule to quote: **balances plus money in flight always equal the total money in the system, and a retry never changes that total.**

9. Resilience: surviving a bad dependency

9.1 The two disasters

- **Cascading failure.** A calls B. B slows down. A's threads pile up waiting, A starts timing out, and A's callers pile up too. One slow part drags the whole chain down.
- **Retry storm.** B has a short outage. Thousands of clients retry at the same instant when it returns, and the surge knocks it down again.

9.2 The four tools

Tool	Meaning	Analogy	Our setting
Timeout	Never wait forever	Hanging up after a few rings	2 s per call, 1 s to connect

Tool	Meaning	Analogy	Our setting
Retry with backoff and jitter	Try again, waiting longer each time, with a random wobble	Redialling a busy number after 1 min, 2, 4, each a bit random	Up to 4 attempts, 200 ms start, doubling, 50% randomness
Circuit breaker	After repeated failures, stop calling for a while and fail fast	A fuse that trips so the house does not burn	Opens at 50% failures over the last 6 calls (min 4 calls)
Bulkhead	Give each dependency its own limited capacity	Watertight compartments in a ship	Max 10 concurrent provider calls

9.3 The circuit breaker's three states

State	Behaviour	Moves on when
CLOSED	Calls flow normally; failures are counted	Failure rate reaches 50%
OPEN	Calls are rejected instantly (fail fast); the provider can recover	After 10 seconds
HALF_OPEN	A couple of test calls are allowed	Tests succeed: back to CLOSED. Tests fail: back to OPEN

Watch it live at `GET /wallets/provider-status`.

9.4 How the tools are layered

```
Supplier<String> guarded = Retry.decorateSupplier(retry,
    CircuitBreaker.decorateSupplier(breaker,
        Bulkhead.decorateSupplier(bulkhead, call)));
```

Read from the inside out: the call is limited by the bulkhead, watched by the breaker, and retried on failure. Retries do **not** happen when the breaker is open or the bulkhead is full, because those mean "stop", not "try again". When everything fails, the controller returns **503 PROVIDER_UNAVAILABLE** instead of hanging.

9.5 Why retrying a payment is safe here

Retrying a charge is normally dangerous because you might charge twice. It is safe here because we send our own operation ID (walletId:Idempotency-Key) as the provider's Idempotency-Key, and the mock provider remembers it: the same key always returns the same chargeId. This is idempotency layer 3. **Retries are only safe on idempotent operations.**

9.6 The mock provider is deliberately nasty

MockProviderController records the charge *before* it may fail, to copy the worst real case: "charged, but the answer never arrived". Two switches:

- **failPercent** (0 to 100): share of calls that return HTTP 500.
- **delayMs**: adds delay, to trigger timeouts.

`GET /mock-provider/stats` shows distinctCharges. After a storm of retries it must equal the number of distinct keys. That proves no double charge.

9.7 Jitter elsewhere

ConflictRetry (ledger write collisions) also waits a random 5 to 25 ms times the attempt number. Same reason: competing requests must not retry in step.

9.8 What to demonstrate

Switch	You do	You see
failPercent=60	Send several card top-ups with new keys	Some succeed after retries, some end 503. distinctCharges equals the number of keys
failPercent=100	Keep sending	provider-status becomes OPEN; later calls fail instantly
failPercent=0, wait 10 s	Send one call	HALF_OPEN, then CLOSED after success

10. Observability, chaos and testing

10.1 Observability

Being able to understand what a running system is doing from the outside, using logs, metrics and traces. *Analogy:* the tracking number on a parcel. Without it you only know "it left" and "it never arrived".

Tool	What we have	Where
Trace ID	One ID per request. Taken from the X-Trace-Id header (letters, digits, _ and -, up to 64 chars) or generated. It goes into log lines, the response header and the trace_id column of every ledger event	TraceFilter, LedgerService
Structured log pattern	Every log line shows [trace=...]	application.yml
Metrics	Counter wallet.duplicates.suppressed, tagged layer=api or layer=inbox	Actuator: /actuator/metrics/...
Saga traceability	The saga passes transfer- as the trace on the ledger events it causes	CommandConsumer

Honest limit: we have trace IDs, not a full tracing tool. OpenTelemetry with Jaeger would draw a timeline across services (optional).

10.2 Chaos engineering

Break the system on purpose, in a controlled way, to prove it survives. *Analogy:* a fire drill.

Switch	Where	What it simulates	What it proves
dropResponse	POST /admin/chaos?dropResponse=true	Server commits, then the answer is lost (HTTP 503)	A retry with the same key is a replay: money moved once
duplicatePublish	.../chaos?duplicatePublish=true	Relay publishes every message twice	The inbox skips the duplicate; the counter rises
failConsumer	.../chaos?failConsumer=true	A consumer that always throws	Retries, then messages land in the DLQ
failPercent, delayMs	/mock-provider/chaos	A flaky or slow payment provider	Timeouts, retries, breaker, one charge per key

These endpoints are for demos only. A real system would never expose them, and the default RabbitMQ login (guest) is also for local use only.

10.3 Testing

Component testing checks a piece in isolation. **End-to-end tests** check the pieces together. The lecture also covers stubs, drivers, mocks, boundary values, equivalence partitions, decision tables and state-transition testing.

Test	Kind	What it checks
RequestHasherTest	Unit	Same input gives same hash, different input a different one, length is 64

Test	Kind	What it checks
IdempotencyE2ETest: 50 parallel identical top-ups	End-to-end, concurrency	Exactly one ledger event; 49 replays; balance 1000
IdempotencyE2ETest: same key, different amount	End-to-end	422, balance unchanged
IdempotencyE2ETest: withdrawal beyond balance	End-to-end	409
IdempotencyE2ETest: 8 parallel different keys	End-to-end, race	Balance 800 and lastSeq 8: no lost update
TransferE2ETest: success	End-to-end, saga	COMPLETED; balances 600 and 400; same key returns same transfer
TransferE2ETest: no funds	End-to-end, saga	FAILED, nothing moves
TransferE2ETest: missing destination	End-to-end, compensation	COMPENSATED, source balance restored
k6/duplicates.js	Load	50 virtual users, one key: final balance exactly 1000

End-to-end tests run with `-De2e=true` and need the services running. Plain `mvn test` skips them.

10.4 Honest assessment: the weakest part for marks

The lecture asks for isolated component tests with stubs and mocks. Today the project has one real unit test class and a strong set of end-to-end tests. An examiner will notice. These additions are cheap and score well:

Add this test	Technique from the lecture
IdempotentExecutor with a mocked JdbcTemplate: the five outcomes in the table in 6.2	Decision table, mocks
ConflictRetry: succeeds on attempt 3; throws TooMuchContentionException after the maximum	Boundary values
LedgerService.append: amount equal to balance passes, balance minus one fails, zero amount rejected	Boundary value analysis, equivalence partitions
SagaService.onReply: every row of the transition table, plus a duplicate reply that must change nothing	State-transition testing
ProviderClient against a stubbed HTTP server: breaker opens after repeated 500s	Stubs
Random mixed operations never break "sum of balances = top-ups minus withdrawals"	Invariant (property-style) testing

11. Java and Spring Boot elements

You know Spring Boot basics, so this ties each feature to the **reason** we use it. The reason is what a viva question usually asks.

11.1 The framework features

Element	Plain meaning	Why it is here
Spring Boot starters, auto-configuration	Add a dependency and Boot configures it	Seven dependencies give a web server, database access, migrations, a broker client and metrics

Element	Plain meaning	Why it is here
Dependency injection	Spring builds objects and hands them what they need	Small, replaceable services, so tests can pass in fakes
@RestController, @PostMapping	Turn methods into HTTP endpoints	The public REST API
@RequestHeader	Reads an HTTP header	Reads Idempotency-Key and X-Trace-Id
Bean Validation (@Valid, @Positive)	Declarative input rules	Rejects zero or negative amounts before any logic runs
Java record	Compact immutable data holder	Request bodies and small results (Stored, Row)
@RestControllerAdvice	One place that maps exceptions to HTTP responses	422 key reuse, 409 no funds, 404 missing wallet
@Transactional	Wrap a method in a database transaction	The heart of the project: key, effect and outbox commit together
Transaction propagation	How a method joins an existing transaction	MANDATORY makes LedgerService.append refuse to run outside one
JdbcTemplate	Plain SQL with little ceremony	We need ON CONFLICT, SKIP LOCKED and exact control
@Scheduled	Run a method on a timer	Outbox relay, key cleanup, saga recovery
Spring AMQP (RabbitTemplate, @RabbitListener)	Send and receive RabbitMQ messages	Publishing outbox rows; consuming events, commands, replies
Listener retry settings	Retry a failing consumer, then reject	Feeds the dead-letter queue
OncePerRequestFilter	Code that runs once around every request	Assigns the trace ID (TraceFilter)
MDC (logging context)	Key/value pairs added to every log line	Puts traceId in the logs
Actuator and Micrometer	Built-in health and metrics endpoints	The duplicates-suppressed counter
Flyway	Runs versioned SQL at startup	Same tables on every machine
Maven	Dependency and build tool	Two independent projects, one pom.xml each
Docker Compose	Describes several containers in one file	PostgreSQL and RabbitMQ with docker compose up -d

11.2 Java details worth knowing

- **Lambdas and Supplier<T>.** executor.execute(..., Supplier<Stored> effect) takes "the work to do" as a parameter. The executor decides whether and when to run it.
- **volatile fields in ChaosState.** A change by one thread (the admin call) must be visible to other threads (request handlers). volatile guarantees that.
- **ConcurrentHashMap.computeIfAbsent** in the mock provider: an atomic "create if missing", so two requests with one key get one chargeId.
- **ThreadLocalRandom** gives jitter without sharing a lock between threads.
- **java.net.http.HttpClient** is the built-in HTTP client, so no extra library is needed.
- **jakarta.* packages.** Modern Spring uses jakarta.validation and jakarta.servlet, not the old javax.*.

11.3 Why JdbcTemplate and not JPA/Hibernate

JPA hides SQL behind objects, which is great for ordinary CRUD. Here the SQL *is* the design: ON CONFLICT DO NOTHING, UNIQUE (wallet_id, seq), FOR UPDATE SKIP LOCKED, guarded UPDATE ... WHERE state = ?. With JdbcTemplate every statement is visible, explainable and testable. Cost: more hand-written SQL. Say the trade-off was chosen deliberately.

11.4 Spring traps this project avoids (and one it accepts)

- **@Transactional works through a proxy.** Calling a transactional method from another method in the same class skips the proxy and the transaction. That is why IdempotentExecutor is its own bean.
- **A poisoned transaction cannot continue.** After a duplicate-key error it must roll back. So retries happen outside the transactional method (ConflictRetry).
- **Rollback-only marking.** Catching a business exception inside a transaction and carrying on can raise UnexpectedRollbackException. CommandConsumer checks business rules with plain reads first.
- **Rollback rules.** Spring rolls back on unchecked exceptions by default, not checked ones. Our domain errors extend RuntimeException.
- **Accepted shortcut.** Some JSON responses and inter-service messages are built as strings (key=value&key=value). Easy to read, but production would use typed objects and a schema.

12. Project manager view

A project manager asks five questions: what are we building, who does what, when, what could go wrong, and how do we know it is done. Dates are in weeks because the real submission date is unknown.

12.1 Charter

Item	Statement
Objective	Show a working distributed wallet whose money operations are safe under retries, duplicates and crashes
Success criteria	All 11 demo scenarios pass live; the 50-parallel-duplicate test shows exactly one effect; every member can explain every layer in the viva
In scope	Wallets, top-up, withdrawal, card top-up, transfer saga, outbox/inbox, DLQ, resilience, chaos switches, tests, report
Out of scope	Real bank or payment integration, real authentication, a polished UI, cloud deployment
Stakeholders	The lecturer (client and assessor), the viva panel, the 8 group members
Constraints	8 people with uneven Java experience; a fixed deadline; limited AI-assistance usage; one stack (Java, Spring Boot)
Assumptions	Every laptop can run Docker; the README code compiles after small fixes; the viva may be individual

12.2 Roadmap (4 weeks, 3 gates)

Week	Phase	Work	Gate at the end
1	Foundations	Agree contracts, repo, CI, Compose, Flyway tables, name role owners	Gate 1: contracts agreed
2	Thin slice	Idempotent top-up, ledger + outbox, relay to RabbitMQ, inbox consumer	Gate 2: slice demo passes
3	Distribute	Transfer saga, refund + recovery, provider + breaker, chaos switches	Gate 3: saga + failures pass

Week	Phase	Work	Gate at the end
4	Prove and polish	Feature freeze, tests + k6, two viva rehearsals, report + slides	Done

A **gate** is a go/no-go check. If the group does not pass it, next week's work does not start. The **feature freeze** at the start of week 4 means only tests, fixes, the report and rehearsals are allowed. A smaller system that survives every demo beats a bigger one that only works on the happy path.

12.3 Work breakdown and ownership

#	Role	Deliverable	Done when
1	Idempotency lead	idempotency/*, WalletController, error handling	The 6.2 outcome table passes as tests
2	Ledger lead	LedgerService, V1__init.sql, replay endpoint	Overdraft race test passes; replay says consistent
3	Saga lead	SagaService, TransferController, recovery job	Success, no-funds, compensation and crash scenarios pass
4	Messaging lead	RabbitMQ topology, relay, consumers, DLQ	Duplicate delivery skipped; poison message reaches the DLQ
5	Resilience lead	ProviderClient, mock provider, card top-up	Breaker opens and closes; one charge per key
6	Chaos and load lead	Chaos switches, k6, demo runbook	Demo runs end to end in under 10 minutes, twice
7	Quality lead	Unit and component tests, coverage, CI	The extra tests in 10.4 exist and pass
8	Observability, docs, slides lead	Trace ID, report, slides, screenshots	Report matches the code; every claim has a screenshot

Each owner is also the viva expert for their part, but everyone must be able to explain the whole flow.

12.4 Dependencies and critical path

The longest chain that cannot be shortened: **agreed contracts -> ledger -> idempotent executor -> outbox and relay -> saga -> chaos demo.** If the ledger slips, almost everything slips, so the ledger lead should be your strongest Java developer. Work that does not block the chain: the mock provider, k6 scripts, the report skeleton, and tests written against the agreed contracts.

12.5 Risk register

Risk	Likelihood	Impact	Mitigation	Owner
README code has compile errors (never compiled by its author)	High	Medium	Fix on day one with one person; share the fixed commit	1
Spring Boot 4 API differences surprise the team	Medium	Medium	Pin the version; search the error with "Spring Boot 4 migration guide"	1
Uneven Java skill leaves one person carrying the work	High	High	Pair weak members with role owners; rotate who runs the demo	all
Individual viva exposes members who built only one part	High	High	Two rehearsals where each member explains a part they did not build	7
Docker or port problems on some laptops	Medium	Medium	Shared setup-check script; a fallback machine for the demo	6

Risk	Likelihood	Impact	Mitigation	Owner
End-to-end tests are timing-sensitive and flake	Medium	Medium	Use the polling helper; no fixed sleeps; run twice before demo day	7
Scope creep (UI, extra features)	Medium	High	Feature freeze gate; a "parking lot" list for ideas	PM
Demo fails live	Medium	High	Rehearse twice; keep a recorded backup of every scenario	6
Attribution issue with the reference repository	Low	High	State in the report that the idea was inspired by it and the code is a new Java design	8
AI-assistance limits reached near the deadline	Medium	High	Spend the budget early on the critical path; ask for small diffs	PM

12.6 Definition of done

- Both services start with docker compose up -d and two mvn runs, with no manual database edits.
- All 11 demo scenarios pass on a clean machine.
- mvn test passes; mvn test -De2e=true passes with both services running.
- The extra component tests from 10.4 exist.
- The report matches the code (names, ports, tables) and includes diagrams and screenshots.
- Every member has explained every layer once, out loud, to another member.
- The limitations in section 13 are written into the report.

12.7 What examiners probably reward

This is an informed guess from the slides. Check it against the real assessment brief.

Likely criterion	Evidence we show
Understanding of Idempotency and Exactly-Once(ish)	Three layers, the same-transaction rule, honest wording about "ish"
Use of distributed design patterns	Outbox, inbox, saga, CQRS, event sourcing, resilience
Working system and demo	11 live scenarios, failure first then the fix
Testing quality	Unit, component, end-to-end and load tests
Design justification	Why saga not 2PC, why optimistic concurrency, why JdbcTemplate
Teamwork and individual understanding	Roles, Git history, viva rehearsals

12.8 Ways of working

- **Cadence:** two 15-minute stand-ups a week (done, next, blocked), plus the weekly gate review.
- **Git:** one repository, protected main, a feature/<role> branch each, merge by pull request with one reviewer.
- **Decision log:** a short file listing each design decision, the options and the reason. It becomes your report's justification section almost for free.
- **Measures:** demo scenarios passing (target 11 of 11), tests passing, duplicates suppressed in the 50-parallel run (target 49).

13. Viva playbook, limitations, glossary, study plan

13.1 Strategy

- **Lead with the problem, not the technology.** Panels remember the lost-reply story better than a list of patterns.
- **Show failure first, then the fix.** Switch idempotency off, double-charge, then switch it on.
- **Say "effectively-once", never "exactly-once".** Explain why "ish" is honest.
- **Name your own limits before they do.** It reads as maturity, not weakness.
- **Answer in your own words.** If you do not know, say how you would find out. Do not bluff.
- **Point at code.** For each answer, open the one file that proves it.

60-second pitch: "Our system is a digital wallet. The problem is that networks lose replies, so a customer's retry can move money twice. We solved it by giving every request an idempotency key and saving the key, the ledger event and the outgoing message in a single database transaction. Messages travel through an outbox to RabbitMQ and are de-duplicated by an inbox on the other side. Transfers between wallets run as a saga with refunds. To prove it, we break the system on purpose: we drop responses, duplicate messages, crash a service and flood it with fifty identical requests, and the money always adds up."

13.2 Harder viva questions

Question	Strong answer
Isn't this just a unique constraint?	A unique constraint stops the duplicate but gives the client an error. We also store and replay the original response, check the payload hash, and make the effect and the record atomic.
What if the idempotency table is lost?	Replays become new operations. That is why the ledger has UNIQUE (wallet_id, seq) and saga commands have their own keys: defence in depth. In production the table lives in the same durable database as the ledger.
Who generates the key?	The client, once per intention (a UUID per "Send" tap), reused on every retry. A server-generated key would differ each attempt and defeat the purpose.
What if a client uses a weak or guessable key?	Keys are scoped to endpoint and wallet, and a reused key with a different payload is rejected. In production, add the user ID to the scope.
What happens at 24 hours?	The row is deleted and a very late retry is treated as new. The TTL must exceed any realistic client retry window.
Why RabbitMQ and not Kafka?	We need task-style queues, routing keys, acknowledgements and dead-letter queues, which RabbitMQ gives out of the box. Kafka suits replayable high-volume event logs and would be heavier than needed.
Why not store keys in Redis?	Redis is a different system from the ledger, so the key and the effect could not commit atomically. That would bring back the exact gap we closed.
Is outbox plus inbox overkill?	Each solves a different failure: the outbox stops a lost message on publish; the inbox stops a double effect on delivery. Remove either and a failure case comes back.
Why return 202 for a transfer?	The saga is asynchronous. 202 means "accepted, still in progress"; the client polls the transfer state.
How would it scale?	Ledger: partition wallets across instances (one writer per wallet). Relay: SKIP LOCKED with several instances. Consumers: competing consumers on the queue.
What if two relays publish the same row?	SKIP LOCKED prevents it in normal running. If it ever happened, the inbox dedupes it.
How do you know the total money is safe?	The invariant test: balances plus money in flight equal top-ups minus withdrawals, after every demo scenario.

13.3 Honest limitations (say these first)

Limitation	Effect	How to phrase it	Fix
No publisher confirms on the relay	A broker crash at the wrong moment could lose one message	"Delivery is at-least-once from the relay, but we do not yet use confirms."	Enable publisher confirms; mark PUBLISHED only on confirm
A replayed saga command sends no second reply	A lost reply could leave a transfer in DEBITED	"Recovery re-sends commands, but a stored result does not re-send its reply."	On a replay, write the reply to the outbox again
A replayed card top-up still calls the provider first	A replay fails if the provider is down	"The provider is safe to call twice, but we could skip it."	Check the stored key before calling the provider
Balance projection updated synchronously	Always consistent, so simpler than true CQRS	"A deliberate simplification."	Update it asynchronously from the outbox stream
Keys not tied to a user	No authentication exists	"Out of scope."	Add the user ID to the key scope
Messages are key=value strings	Fragile and untyped	"Chosen to avoid libraries."	JSON or Avro with a schema
Chaos endpoints open; RabbitMQ default login	Insecure outside a demo	"Demo only."	Remove or protect them
No automatic replay of dead-letter messages	A human must act	"Operational follow-up."	A small admin job to inspect and republish
No snapshots	Replay slows on very long histories	"Not needed at this scale."	Snapshot every N events
Component tests are thin	Weaker marks on the testing lecture	Fix it, do not explain it	The additions in 10.4

13.4 Glossary

Term	Plain meaning
Idempotent	Doing it many times has the same effect as doing it once
Idempotency key	A client-chosen ID that names one intention, reused on every retry
Effectively-once	At-least-once delivery plus idempotent handling, so the effect happens once
Event sourcing	Store the history of facts, work out the state from it
CQRS	Separate model for writing and for reading
Projection	A read model built from events
Replay	Rebuilding state by re-applying events
Optimistic concurrency	Detect a clash when saving instead of locking first
Transaction	A group of changes that succeed or fail together
Outbox	A table of messages committed with the data, published later
Inbox	A table of processed message IDs used to skip duplicates
Relay	The worker that moves outbox rows to the broker
Dead-letter queue	A holding queue for messages that keep failing
Poison message	A message that always fails when processed
Saga	A chain of local transactions with compensations
Compensation	A new action that undoes the business effect of an earlier step

Term	Plain meaning
Orchestration	One coordinator directs the saga
Circuit breaker	Stops calls to a failing dependency for a while
Bulkhead	Limits capacity per dependency so one cannot starve the rest
Jitter	Random variation in retry delay
Retry storm	Many clients retrying together and overloading a recovering service
Trace ID	An ID that follows one request through logs and events
Chaos engineering	Breaking things on purpose to prove resilience

13.5 Study plan

Five sessions of about 45 minutes. Each ends with a whiteboard drill: explain out loud with no notes.

Session	Read	Whiteboard drill
1	Sections 1 and 2	Draw the lost-reply timeline and explain why a retry is unavoidable
2	Sections 4 and 5	Draw the overdraft race and how the unique constraint resolves it
3	Section 6	Draw the three layers and walk through 50 parallel identical requests
4	Sections 7 and 8	Draw outbox -> relay -> broker -> inbox, then the saga state machine with a refund
5	Sections 9 to 13	Run the full demo once, then answer ten random questions from 13.2

When you can pass every drill without looking, you are ready. A good test: explain one layer you did not build to a teammate, who then tries to find a hole in it.